

Jake's website: deploy to AWS and sign in safely

Website documentation for jakeardoin.com

Prepared for the Django, Wagtail, and AWS website setup.

Keep this PDF with your password manager and deployment notes. The Markdown file remains the editable source.

Jake's website: deploy to AWS and sign in safely

These instructions use the `codex/fresh-wagtail-website` branch of `GreatestJakeEVR/mywebsite` and replace the earlier Lightsail plan. No AWS resources have been created yet. Creating the stacks below starts AWS charges.

Your homepage, blog, and cookbook remain public. Editing requires a password plus a physical security key. You will register and test your own YubiKeys; automated tests cannot perform that physical step.

1. Know what you will be using

Service	What it does
EC2	The computer rented from AWS that runs your site.
PostgreSQL	Stores pages, recipes, drafts, users, and registered key public credentials.
Wagtail	The browser interface where you write and publish content.
S3 media bucket	Stores original uploads and generated image sizes.
CloudFront	Delivers the generated public images.
Caddy	Provides HTTPS certificates and directs requests to Django.
S3 backup bucket	Stores private daily database backups.
GitHub	Stores code, design files, and deployment instructions.

Writing a post in Wagtail does not require deploying code. Changing the layout or adding functionality does. Wagtail content lives in the database, not GitHub. Your local database and production database are separate; publishing locally does not publish online.

This is a single-server deployment: the application and database run on one EC2 instance. An instance outage takes the site offline until repaired/restored. It is simpler than running a load balancer and a separate managed database, which can be added later.

2. Prepare two keys and your accounts

- 1 Have your AWS account, GitHub account, and domain-provider sign-in available. The domain provider is where you bought `jakeardoin.com`; DNS may be managed by another provider.
- 2 Use a current Chrome or Edge browser. Have two FIDO2-capable YubiKeys: one everyday key and a backup stored separately. An older OTP-only key will not meet this setup's requirements. Check your model against [Yubico's specifications](#).
- 3 Save a different strong password for each account in your password manager.
- 4 When the browser offers Windows Hello, a phone, or a password-manager passkey, choose **Security key**, sometimes under **Use another device** or **More choices**. Insert the YubiKey, enter its security-key PIN if requested, and touch it.
- 5 The key PIN is different from your account password. If prompted to create a PIN, save it. Do not reset an already-used key to fix a forgotten PIN; resetting erases credentials.
- 6 Register both keys separately with each service. Name them `Jake everyday YubiKey` and `Jake backup YubiKey`. Test each in a fresh sign-in before storing the backup.

The editor requests an external key and rejects backup-eligible synced passkeys. It does not verify Yubico manufacturer attestation; enroll your actual YubiKeys. AWS's security-key option also allows built-in authenticators, so choose your physical keys explicitly there too.

3. Protect the AWS root account

- 1 Open [AWS Console](#). Create an AWS account first if needed, completing email, payment, and identity verification.
- 2 Sign in as **Root user** using the email address that owns the account.
- 3 Open the account menu at the upper right → **Security credentials**.
- 4 Under **Multi-factor authentication (MFA)**, choose **Assign MFA device**.
- 5 Name the everyday key, select **Passkey or security key**, and complete the browser's external-key prompt.
- 6 Repeat for the backup key.
- 7 Sign out and test each key in a fresh sign-in. Keep root for initial setup and recovery; create the daily sign-in below.

AWS supports multiple registered keys. See [AWS's security-key instructions](#).

4. Create your everyday AWS administrator login

These steps assume a new personal AWS account with no existing company identity provider. If an organization already manages your AWS account, ask its administrator to assign access instead of replacing identity settings.

- 1 Search for **IAM Identity Center** in AWS. Select **US East (N. Virginia), us-east-1** as the region for this guide.
- 2 Choose **Enable** and select an **organization instance**, not an account instance. If prompted to create an AWS Organization, accept that for this new-account setup. Organization instances can assign AWS account access. [AWS explains the instance types here](#).
- 3 Keep the built-in **Identity Center directory** as the identity source.
- 4 Open **Users** → **Add user**. Create `jake-admin`, enter your name/email, choose email invitation/password setup, and finish.
- 5 Open **Permission sets** → **Create permission set**. Choose the predefined **AdministratorAccess** permission set. Set its session duration to **1 hour**, then create it. This is your infrastructure administrator; do not give this permission to other blog editors.
- 6 Open **AWS accounts**, select your account → **Assign users or groups**. Select your user, then the **AdministratorAccess** permission set, and submit.
- 7 Open **Settings** → **Authentication** → **Multi-factor authentication** → **Configure**.
- 8 Choose **Every time they sign in (always-on)**. [Prompt settings](#).
- 9 Enable **Security keys and built-in authenticators** and disable **Authenticator apps** for this intended key-based account. [MFA types](#).
- 10 Initially choose **Require them to register an MFA device at sign in** for users without a device. Enable **Users can add and manage their own MFA devices**, then save. [Self-registration settings](#).

- 11 Complete the invitation email. Bookmark the **AWS access portal URL** displayed in Identity Center; it usually ends in `awsapps.com/start`.
- 12 Open that portal in a private browser window. Sign in with your new password and register the everyday YubiKey when prompted.
- 13 In the portal's user menu, open **Security credentials / My security credentials**. Add the backup key. Sign out and test both keys separately.
- 14 Return to Identity Center's MFA configuration and change the setting for users without a registered device to **Block their sign-in**. Keep **always-on**. Save. Enroll future AWS administrators before applying this rule to them. [Device enforcement](#).
- 15 Sign out of root. From now on, open your bookmarked portal, select your AWS account and **AdministratorAccess**, and enter the console that way.

Do not create personal AWS access keys for this guide. Existing API keys and alternate administrator accounts can bypass this browser policy. Review **IAM** → **Users** → **Security credentials** and retire unused credentials after verifying your new login. The server's limited machine role is separate and has no console password.

5. Protect GitHub, domain management, and email

- 1 In GitHub, open your profile menu → **Settings** → **Password and authentication**.
- 2 Enable 2FA if necessary. Complete GitHub's initial setup and store recovery codes securely.
- 3 Under **Security keys**, select **Register new security key**. Register both YubiKeys and test them. Other configured recovery/2FA methods may remain; registering a key does not remove them. [GitHub instructions](#).
- 4 In your DNS/domain and email providers, open account **Security** settings and register the keys wherever FIDO2/WebAuthn is supported. Save recovery information securely. Exact menus depend on your providers.
- 5 Review existing GitHub tokens, SSH keys, connected apps, and domain-provider API tokens. Keep only needed, restricted credentials. Browser MFA does not make an already-issued API token require a key touch for each use.

If a domain/email provider lacks security-key support, that part of the requested protection remains incomplete until you use a supporting provider. Website code cannot enforce another company's login policy.

6. Set a spending alert

- 1 Search **Billing and Cost Management** → **Budgets** in AWS.
- 2 Choose **Create budget** and a monthly cost budget. Pick a limit you are comfortable with, for example **\$60**, and enter your email.
- 3 Enable actual and forecasted alerts and save. Alerts notify you; they do not cap charges.
- 4 Review the setup in [AWS Pricing Calculator](#): Linux `t3.medium`, `us-east-1`, 730 hours/month, 30 GB `gp3` disk, one public IPv4 address, S3 storage/requests, CloudFront traffic, and SES if used. The example \$60 alert is not a price quote. Traffic, CPU burst credits, backups, and taxes affect cost.

7. Download the deployment templates

- 1 Open [your website branch](#).

- 2 Check that the branch selector reads `codex/fresh-wagtail-website`.
- 3 Choose **Code** → **Download ZIP**.
- 4 In Windows File Explorer, right-click the downloaded ZIP → **Extract All**.
- 5 Open the extracted folder, then deploy. You will upload `media.cloudformation.yaml` and `server.cloudformation.yaml` below.

The existing local project also contains these files in `deploy`. Do not upload `.env.production`; that file contains private configuration.

8. Create image storage

CloudFormation creates AWS resources from a template so you do not need to configure each permission manually.

- 1 Enter AWS through your protected access portal. Select **us-east-1** at the upper right.
- 2 Search **CloudFormation** → **Create stack** → **With new resources (standard)**.
- 3 Choose **Choose an existing template** → **Upload a template file**. Upload `media.cloudformation.yaml` and continue.
- 4 Name the stack `jake-website-media`. Keep default stack options, review, and choose **Submit**.
- 5 Refresh its **Events** tab until status is **CREATE_COMPLETE**. CloudFront can take several minutes. If it fails, read the first failed resource's reason before retrying.
- 6 Open **Outputs**. Save `MediaBucketName`, `CloudFrontDomain`, and `Region` in a setup note.

The bucket remains private. CloudFront can read generated images/ files through origin access control. Do not enable public bucket access. [AWS's origin access control documentation](#).

9. Create the server and backup storage

- 1 Create another standard CloudFormation stack and upload `server.cloudformation.yaml`.
- 2 Name it `jake-website-server`.
- 3 Paste the previous stack's **MediaBucketName** into that parameter. Leave the Ubuntu image parameter unchanged.
- 4 Continue to review, acknowledge IAM resource creation, and submit.
- 5 Wait for **CREATE_COMPLETE**. Save `InstanceId`, `StaticIPv4`, and `BackupBucketName` from **Outputs**.
- 6 Search **EC2** → **Instances**. Select `jake-ardoin-website` and wait for status checks to pass.

The template creates its own network, a 4 GB server, an encrypted 30 GB disk, a stable public address, and private backup storage. Only web ports 80/443 are exposed. SSH, PostgreSQL, and Django's internal port stay closed to the internet. A limited instance role supplies temporary S3 credentials and Session Manager access.

Accidental termination protection is enabled. The disk is retained if the server is eventually terminated; both buckets are retained if stacks are deleted. Retained resources continue to incur charges until deliberately cleaned up.

10. Open the server's browser terminal

- 1 In **EC2** → **Instances**, select your instance → **Connect**.
- 2 Open the **Session Manager** tab → **Connect**.
- 3 If it is not ready, wait a few minutes and refresh. Preserve the template's instance role and outbound internet access; do not open SSH as a workaround.
- 4 Enter each command separately and press Enter:

```
sudo -i
cloud-init status --wait
docker --version
docker compose version
```

`sudo -i` gives this terminal permission to configure the server. `cloud-init` waits for initial software installation; it should report done. If it reports an error, inspect `tail -n 80 /var/log/cloud-init-output.log` and fix that first. The Docker commands should print version numbers.

All bash boxes below run in this AWS browser terminal, not Windows PowerShell, unless specifically stated otherwise. On each later reconnection, enter `sudo -i` and `cd /opt/mywebsite` again.

Session Manager uses AWS permissions without an inbound SSH connection. MFA protects sign-in; the resulting session remains usable until it expires or you sign out, rather than requiring another touch for every command. [AWS Session Manager](#).

11. Download your website onto the server

```
git clone --branch codex/fresh-wagtail-website --single-branch https://github.com/GreatestJakeEVR/
mywebsite.git /opt/mywebsite
cd /opt/mywebsite
git log -1 --oneline
```

The last line shows the downloaded version. This public repository requires no password/token to read. The server receives no GitHub write access.

12. Create private production settings

- 1 Copy the example and restrict file access:

```
umask 077
cp .env.production.example .env.production
chmod 600 .env.production
```

- 1 Generate secrets directly into the file without printing them:

```
python3 - <<'PY'
from pathlib import Path
import secrets
p = Path('.env.production')
text = p.read_text()
text = text.replace('REPLACE_WITH_A_RANDOM_SECRET_AT_LEAST_50_CHARACTERS_LONG', secrets.token_hex(40))
text = text.replace('REPLACE_WITH_RANDOM_HEX', secrets.token_hex(32))
p.write_text(text)
PY
```

- 1 Open the file with `nano .env.production`.
- 2 Keep `SITE_HOSTNAME=www.jakeardoin.com`, `APEX_HOSTNAME=jakeardoin.com`, and `SITE_URL=https://www.jakeardoin.com`.
- 3 Replace `REPLACE_WITH_MEDIA_BUCKET` with the media bucket output.

- 4 Keep `AWS_S3_REGION_NAME=us-east-1`.
- 5 Replace `REPLACE_WITH_DISTRIBUTION_DOMAIN` with the CloudFront hostname, without `https://` or a slash.
- 6 Replace `REPLACE_WITH_BACKUP_BUCKET` with the backup bucket output.
- 7 Email fields can initially stay empty; login works without email, but email password reset needs section 19.
- 8 Save in nano with **Ctrl+O**, **Enter**, then exit with **Ctrl+X**.

Do not add AWS access keys; EC2 supplies temporary credentials. Keep an encrypted copy of this file in your password manager. Never commit it or upload it to public storage. Do not repeat the copy step over an existing production file later: it would replace the saved settings.

13. Point your domain at the server

- 1 Sign in to the provider managing your DNS. Open records for `jakeardoin.com` and export/save the existing records for rollback reference.
- 2 Add/update an **A** record: name `@` (or blank if your provider requires), value **StaticIPv4**, TTL **300 seconds** if available.
- 3 Add/update another **A** record: name **www**, same IP, same TTL.
- 4 Replace an old `www` CNAME if it conflicts with the new A record. Remove stale A/AAAA records only for these two website names if they point elsewhere. This server has no configured IPv6 address, so an old AAAA can send visitors elsewhere.
- 5 Preserve MX/TXT records used by email and other services. You do not need to transfer the domain or change nameservers.
- 6 If the provider offers a proxy setting, use **DNS only** for this setup.
- 7 Save and allow DNS caches to update; the old TTL affects the delay.

To check from **Windows PowerShell on your computer**:

```
Resolve-DnsName jakeardoin.com -Type A
Resolve-DnsName www.jakeardoin.com -Type A
```

Both should return StaticIPv4. These records switch the existing website at those names to this server.

14. Build the website and create your editor account

Back in the **AWS terminal**:

```
cd /opt/mywebsite
bash deploy/release.sh
```

The script builds the app, starts PostgreSQL, saves a pre-migration backup, checks production settings, applies migrations, and starts the site. Wait for `ReLeased . . .`. Resolve any error before continuing.

Caddy obtains and renews HTTPS certificates after DNS resolves and ports 80/443 are reachable. It redirects the bare domain to `www`. [Caddy HTTPS documentation](#).

Create the initial pages and your account:

```
bash deploy/compose.sh run --rm web python manage.py bootstrap_site --recipes --hostname www.jakeardoin.
```

```
com --port 443
bash deploy/compose.sh run --rm web python manage.py createsuperuser
```

Bootstrap creates Home, Blog, and Cookbook, imports the original 66 recipes, and uploads the initial portrait to S3. Omit `--recipes` for an empty cookbook. It does not copy later edits from your local database.

For `createsuperuser`, enter a username such as `jake`, your email, and a strong password twice. Password characters do not appear while typing in the terminal. Save these credentials; there is no default account.

Open [your website](#). Confirm the homepage and HTTPS work. Resolve certificate warnings before entering credentials.

15. Register both YubiKeys for Wagtail

Do this immediately after account creation. Until the first key is registered, the initial account password allows enrollment; keep it private.

- 1 Open [the editor](#) and bookmark this exact `www HTTPS` address.
- 2 Enter the username and password created above.
- 3 The first login opens **Add security key**. Name your everyday key.
- 4 Click Add/Activate, choose the external **Security key** in the browser prompt, insert the YubiKey, provide its PIN if asked, and touch it.
- 5 Complete the subsequent key-verification prompt before entering Wagtail. Registration alone does not grant editor access.
- 6 Open **Settings** → **Security keys**, or [the key list](#).
- 7 Choose **Add security key**, name the backup, and register it. Complete any reauthentication prompt first.
- 8 Sign out. Open a private browser window, enter your password, and sign in with only the everyday key connected.
- 9 Sign out, close that window, open another private window, and repeat using only the backup.
- 10 Store the backup separately. The site refuses to remove your last registered key.

Your website password, AWS password, and key PIN are different. AWS/GitHub sign-in does not automatically sign you into Wagtail.

16. Test real protection before relying on it

- 1 Signed out in a private window, confirm the homepage/blog/cookbook load publicly.
- 2 Open `/admin/`: it must ask for your password and then a key.
- 3 Enter the correct password but cancel the key prompt. `/admin/` and `/admin/images/` must remain inaccessible.
- 4 Repeat using the everyday key, then the backup; both should succeed separately.
- 5 Open `/admin/login/` directly. It must use the same protected login.
- 6 In EC2's **Security** → **Security groups** → **Inbound rules**, confirm only ports 80/443 are open: no 22, 5432, or 8000.
- 7 Sign out of AWS and confirm a fresh access-portal login requires your key.

Automated checks do not replace these physical-key tests. YubiKey protection is not fully deployed until you complete them.

17. Write and publish

Follow [the detailed Wagtail guide](#) for homepage edits, blog posts, recipes, images, drafts, and publishing.

Your usual routine is: visit `/admin/`, password + key, edit, preview, publish, sign out. No AWS deployment is needed for content changes. Fresh password/key verification is required after roughly an hour; save drafts regularly.

18. Turn on backups and test a restore

In the AWS terminal:

```
cd /opt/mywebsite
bash deploy/backup.sh
cp deploy/website-backup.service deploy/website-backup.timer /etc/systemd/system/
systemctl daemon-reload
systemctl enable --now website-backup.timer
systemctl list-timers website-backup.timer
```

The first command must print `Uploaded database/...`. The timer runs daily around 08:00 UTC. The private backup bucket expires daily objects after 30 days; old-version cleanup can take longer. Local copies remain too; monitor disk usage.

Open **S3** → **backup bucket** → **database/** and confirm a dump exists. Check later scheduled runs with:

```
journalctl -u website-backup.service --since '2 days ago'
```

Database backups include content, users, and key public credentials. Images remain in the separately versioned media bucket. Keep `.env.production` encrypted elsewhere and record the code version with backups. GitHub is not a Wagtail-content backup.

Before server upgrades, also use **EC2** → **Volumes** → **select the attached disk** → **Actions** → **Create snapshot**.

Give it a dated name and wait for completion under **Snapshots**. Snapshots preserve server configuration and volumes; database dumps remain the preferred database restore source. Snapshots incur charges.

To test a dump without changing live content, run `ls -lh backups`, copy a filename, then replace `YOUR_BACKUP`. dump below:

```
bash deploy/compose.sh exec -T db createdb -U website website_restore_test
bash deploy/compose.sh exec -T db pg_restore -U website -d website_restore_test --no-owner --exit-on-error < backups/YOUR_BACKUP.dump
bash deploy/compose.sh exec -T db psql -U website -d website_restore_test -c 'SELECT count(*) FROM wagtailcore_page;'
```

Confirm restore success and a plausible page count. Then delete only that temporary database:

```
bash deploy/compose.sh exec -T db dropdb -U website website_restore_test
```

Do not substitute `website` in the last command: that is the live database. See [recovery procedures](#) for off-server download and live restoration.

19. Configure password-reset email with SES

Email never replaces the key. Resetting a password leaves key registrations in place. Until email is configured, you can change passwords from the server as described in section 22.

- 1 Open **Amazon SES** in us-east-1.

- 2 Open **Configuration** → **Verified identities** → **Create identity** → **Domain**. Enter `jakeardoin.com` and enable DKIM.
- 3 Add the DNS records SES shows at your DNS provider. Preserve existing mail-provider MX records. Wait for SES to show the domain as verified. [SES identities](#).
- 4 While SES is in its sandbox, also create an **Email address** identity for your Wagtail email and click its emailed verification link. Sandbox recipients must be verified. Request production access from the SES account dashboard when sending to unverified recipients is needed. [Sandbox rules](#).
- 5 Open **SMTP settings** → **Create SMTP credentials**. Create `jake-website-mail` and save the displayed SMTP username/password securely. These are regional mail credentials, separate from console credentials. [SMTP setup](#).
- 6 In the AWS terminal, open `nano .env.production`. Set `EMAIL_HOST=email-smtp.us-east-1.amazonaws.com`, `EMAIL_PORT=587`, the SMTP `EMAIL_HOST_USER` and `EMAIL_HOST_PASSWORD`, and `DEFAULT_FROM_EMAIL=webmaster@jakeardoin.com`. Save and exit.

- 7 Recreate the web container to read those settings:

```
bash deploy/compose.sh up -d --force-recreate --wait web
```

- 1 Sign out and test [password reset](#). Check inbox/junk mail, complete the reset, and confirm the key remains required.

The SMTP user needs only sending permission, no console password or server permissions. The website offers no email/SMS/TOTP bypass for its security-key check.

20. Deploy future code changes

- 1 Make/test changes locally and save them to the website branch on GitHub.
- 2 In GitHub's **Actions** tab, confirm **Website checks** succeeded for the exact commit you intend to deploy.
- 3 Enter AWS through your key-protected portal and open Session Manager.
- 4 Run:

```
sudo -i
cd /opt/mywebsite
bash deploy/backup.sh
git pull --ff-only origin codex/fresh-wagtail-website
git log -1 --oneline
bash deploy/release.sh
```

- 1 After success, check the homepage, a post, and a recipe; sign into Wagtail and test an image upload.

Code releases preserve PostgreSQL volumes and S3 uploads. Never run `docker compose down -v` on production; it deletes volumes. Do not copy your laptop's database over production to publish code. If Git reports conflicts, stop and inspect the differences instead of using a hard reset. If migrations ran before a release failed, old code may not match the new database; follow the recovery notes instead of blindly reversing migrations.

21. Maintenance and scheduled publication

Monthly: check backups, disk space (`df -h` and `du -sh backups`), and security updates. Review old local backup files before removing them; retain off-server copies.

Before Ubuntu updates, create a database backup and disk snapshot. Then run `apt-get update` and `apt-get upgrade`, review the proposed changes, and accept. If reboot is needed, run `reboot`; the terminal disconnects. Wait for EC2 checks, reconnect, and verify the site. Docker's restart policies bring services back. Update application dependencies through the tested GitHub/release process too.

Immediate Publish needs no scheduler. For scheduled publishing:

- 1 Run `crontab -e` in the root AWS terminal; select nano if asked.
- 2 Add these lines and save with Ctrl+O, Enter, Ctrl+X:

```
* * * * * cd /opt/mywebsite && /usr/bin/flock -n /opt/mywebsite/.deploy.lock /bin/bash deploy/compose.sh
run --rm --no-deps -T web python manage.py publish_scheduled >> /var/log/website-scheduled.log 2>&1
15 8 * * * cd /opt/mywebsite && /bin/bash deploy/compose.sh run --rm --no-deps -T web python manage.py
clearsessions >> /var/log/website-scheduled.log 2>&1
```

- 1 Run `crontab -l` to confirm. Schedule a test draft a few minutes ahead and verify it publishes. Wagtail uses America/Chicago time here. For failures, inspect `tail -n 50 /var/log/website-scheduled.log`. Rotate/archive that log during maintenance.

22. Forgotten password or lost key

Forgot password, still have a key: use email reset, or enter the protected AWS terminal and run:

```
cd /opt/mywebsite
bash deploy/compose.sh run --rm web python manage.py changepassword jake
```

Replace `jake` with your Wagtail username. This leaves keys registered.

Lost everyday key: log in with your password and backup key. Open **Settings** → **Security keys**, add/test a replacement, and remove the lost key. If a session may have been stolen, change the password too, invalidating existing Django sessions.

Lost every website key: use a working AWS key and the AWS administrator portal to open Session Manager. As an explicit emergency action:

```
cd /opt/mywebsite
bash deploy/compose.sh run --rm web python manage.py recover_editor jake --confirm-reset
```

This prompts for a new password, removes the selected user's keys, and signs out every website session. Immediately enroll/test two replacement keys. It is logged and requires server-administrator access; no public webpage or email link performs it.

Lost every AWS key too: use AWS account recovery. Website code cannot recover an AWS account. Keep the backup key away from the everyday key to avoid this situation.

23. Troubleshooting

After `sudo -i` and `cd /opt/mywebsite` in the AWS terminal:

```
bash deploy/compose.sh ps
bash deploy/compose.sh logs --tail=100 web
bash deploy/compose.sh logs --tail=100 caddy
```

Symptom	Check
Domain won't load	A records, stale website AAAA records, EC2 state, inbound 80/443.

Symptom	Check
Certificate fails	Caddy logs, DNS, restrictive CAA records, port 80. Fix DNS before repeated certificate attempts.
502 or unhealthy web	Web logs, database health, missing environment values, migrations.
Image 403	Instance role, bucket/region/CDN values, generated images/ path. Keep S3 public access blocked.
Key prompt missing	Exact <code>https://www.jakeardoin.com</code> address, current browser, physical key selected.
Key request expired	Reload and complete within five minutes; avoid simultaneous login tabs.
Password works but editor won't open	Complete key verification; do not disable the security middleware.
Reset email absent	SES region, identity/recipient verification, SMTP credentials, junk folder, web logs.
Backup failure	Backup-service journal, bucket setting, role, database health.

Application logs record security events with user/key IDs, not passwords. CloudTrail records AWS management API activity; Session Manager shell-output logging is separate and must be configured if you want command transcripts. Restrict logs because terminal activity can contain sensitive settings. Do not share unreviewed configuration or logs publicly.